

CS 112

Introduction to Data Structures

WEEK 04

PyList: Dynamic Arrays

Eric Araújo

Calvin University · Fall 2026

This Week

- 1 Why a plain array isn't enough
- 2 `new` and `delete`
- 3 The stack and the heap
- 4 Building a dynamic array
- 5 Growing when full
- 6 Destructors: now you need one
- 7 Copy constructors: shallow vs deep

KEY

Arrays: The Good and the Bad

Good



- Simple to declare
- Indexing is fast: one multiply and add
- Contiguous, so the cache loves them

Bad



- Size fixed at **compile time**
- Can't grow when you need one more
- No idea how many slots are actually used

This week we will build a Python-like list that grows if needed.

Dynamic Memory Allocation

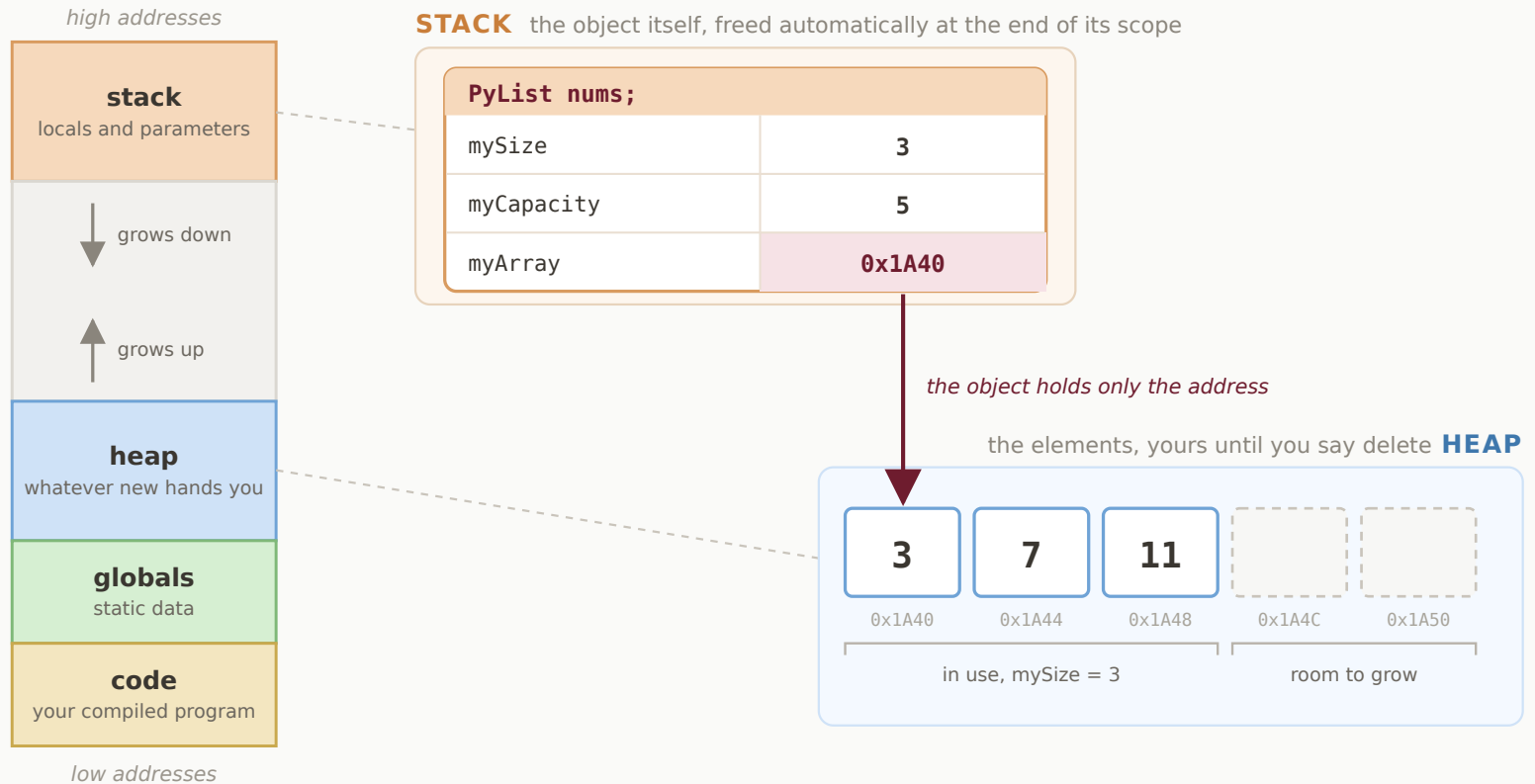
```
int *arr = new int[capacity];    // ask the OS for space
                                   // and a delete to match

arr[0] = 42;                      // use it like an array

delete [] arr;                    // free the space afterwards
arr = nullptr;                   // don't keep the address
```

- `new` returns the **address** of the block
- The size can be a variable: decided while the program runs
- `delete []` for arrays, plain `delete` for single objects

Stack and Heap



The object is on the stack. Its data is on the heap. Only the heap part needs a `delete`.

A Dynamic Array

```
class PyList {
public:
    PyList();                // starts empty, with no array
    PyList(const PyList &original); // copy: builds its own array
    ~PyList();               // frees the array
    unsigned getSize() const; // how many items are in it
    void append(const Item &it); // add one more at the end
    Item& operator[](unsigned i); // v[i], on either side of an =
private:
    Item *myArray;           // address of the block, or nullptr
    unsigned mySize;         // slots used so far
    unsigned myCapacity;     // slots the block has room for
};
```

size is how many items you have. **capacity** is how many fit before it must grow.

Appending

```
void PyList::append(const Item &it) {
    if (mySize == myCapacity) {           // no room left
        myCapacity = (myCapacity == 0)
            ? 1                           // first append
            : myCapacity * 2;             // else double
        Item *bigger = new Item[myCapacity];
        for (unsigned i = 0; i < mySize; ++i) {
            bigger[i] = myArray[i];       // copy each one
        }
        delete [] myArray;                // free the old
        myArray = bigger;                  // keep the new
    }
    myArray[mySize] = it;                  // room now
    ++mySize;
}
```

Growing, Step by Step

- 1 The array is full. There is no room for one more.

```
v.append(9);
```

PyList v;

mySize 4

myCapacity 4

myArray **0x1A40**

0x1A40

3

7

11

5

*mySize == myCapacity
no room for one more*

Growing, Step by Step

2 Ask the heap for a block twice the size.

```
v.append(9);
```

PyList v;

mySize 4

myCapacity 8

myArray **0x1A40**

0x1A40

3

7

11

5

*mySize == myCapacity
no room for one more*

0x2B00

new Item[8]



Growing, Step by Step

3 Copy every item across.

```
v.append(9);
```

PyList v;

mySize 4

myCapacity 8

myArray **0x1A40**

0x1A40

3

7

11

5

*mySize == myCapacity
no room for one more*

0x2B00

3

7

11

5

new Item[8]

Growing, Step by Step

4 Free the old block.

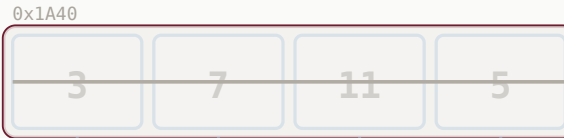
```
v.append(9);
```

PyList v;

mySize 4

myCapacity 8

myArray **0x1A40**



*mySize == myCapacity
no room for one more
delete [] myArray*



Growing, Step by Step

5 Point `myArray` at the new block.

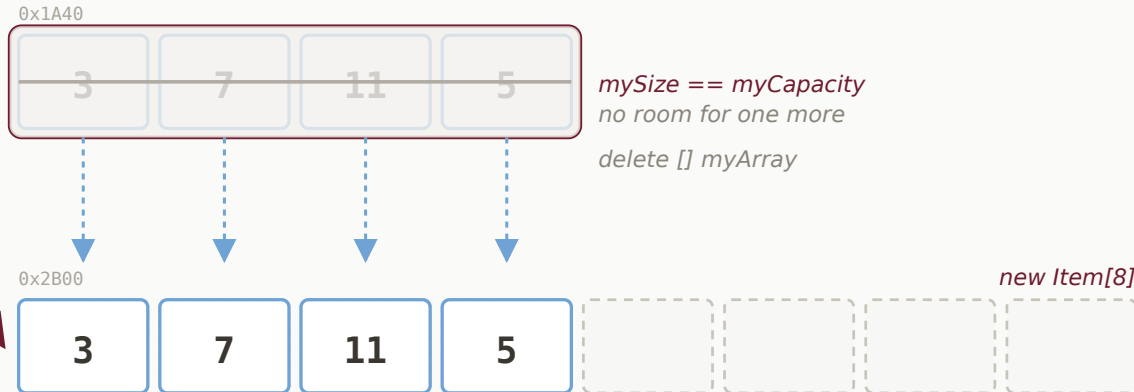
```
v.append(9);
```

PyList v;

mySize 4

myCapacity 8

myArray **0x2B00**



myArray = bigger

Growing, Step by Step

6 Store the new item, and count it.

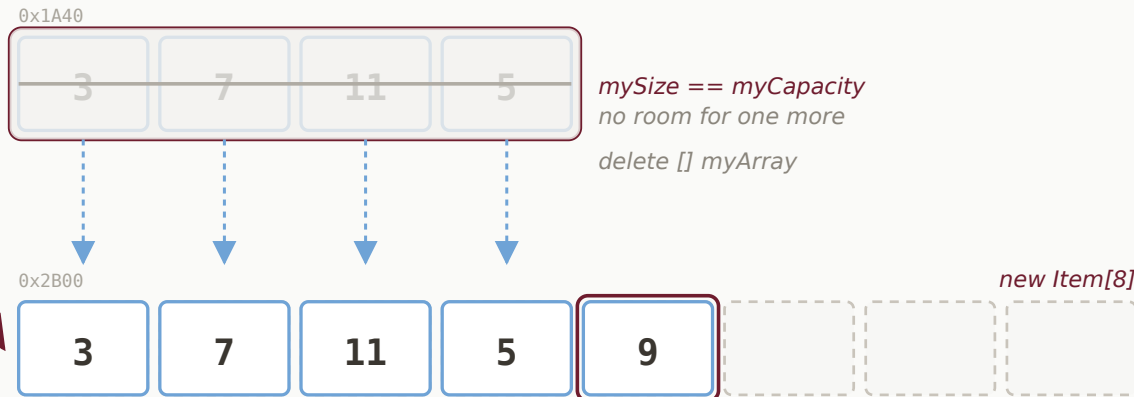
```
v.append(9);
```

PyList v;

mySize 5

myCapacity 8

myArray **0x2B00**



myArray = bigger

TALK TO YOUR NEIGHBOR · TTYN

Why double the capacity instead of adding one slot?

- A.** Doubling uses less memory
- B.** Adding one would make every append copy the whole array
- C.** It doesn't matter, both are equally fast

Destructor

```
PyList::~~PyList() {           // a ~ then the class name
    delete [] myArray;         // give the block back
    myArray = nullptr;         // and don't keep the address
}
```

- Destructors have no arguments or return type. You also never call them.
- They are called when the object loses its scope, and when you delete an object explicitly.
- Only one destructor per class, with no overload permitted.

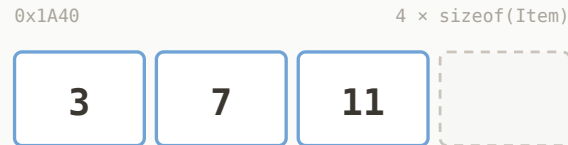
Rule of Thumb

- 1 The function returns, so **v** goes out of scope.

THE STACK

PyList v;	
mySize	3
myCapacity	4
myArray	0x1A40

THE HEAP



} *f() returns*
v goes out of scope

Rule of Thumb

2 The destructor C++ wrote for you runs.

THE STACK

PyList v;	
mySize	3
myCapacity	4
myArray	0x1A40

THE HEAP

0x1A40

4 × sizeof(Item)

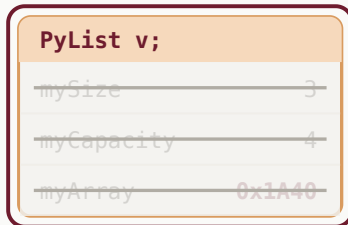


} *f() returns*
v goes out of scope
~PyList() runs
the one C++ wrote for you

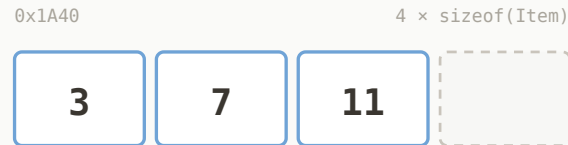
Rule of Thumb

- 3 It destroys the three members. The pointer is one of them.

THE STACK



THE HEAP



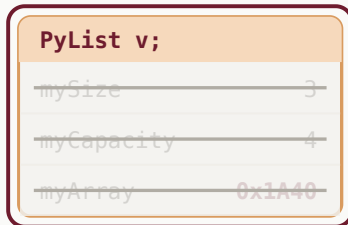
} *f() returns*
v goes out of scope

~PyList() runs
the one C++ wrote for you
all three of them, the pointer included

Rule of Thumb

4 The block is still allocated, and nothing points at it.

THE STACK



}

f() returns

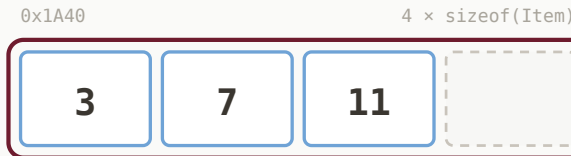
v goes out of scope

~PyList() runs

the one C++ wrote for you

all three of them, the pointer included

THE HEAP



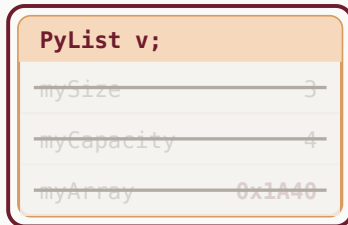
LEAK

*still allocated, and nothing points at it
it stays there until the process ends*

Rule of Thumb

5 The line that frees it has to be yours.

THE STACK



} *f() returns*
v goes out of scope
~PyList() runs
the one C++ wrote for you
all three of them, the pointer included

THE HEAP



delete [] myArray;
in a destructor you write yourself

If your class calls `new`, your class defines a **destructor**. The one C++ writes for you destroys the members, and the block is not a member.

TALK TO YOUR NEIGHBOR · TTYN

In which lines does a **PyList** destructor run?

```
void f() {           // 1
    PyList *p = new PyList; // 2
    PyList v;         // 3
    v.append(3);      // 4
    delete p;         // 5
}                     // 6
```

- A.** Lines 2 and 3, where the two objects are built
- B.** Lines 5 and 6
- C.** Line 6 only, at the closing brace
- D.** Line 5 only, because that is the only **delete**

Copying a PyList

TALK TO YOUR NEIGHBOR · TTYN

What exactly got copied?

```
PyList a;           // mySize 0, myCapacity 0, myArray nullptr
a.append(3);         // a block of 1 on the heap
a.append(7);         // full, so a new block of 2

PyList b = a;        // b is built FROM a
```

- A.** The three members, so `b.myArray` holds the same address as `a.myArray`
- B.** The two items, into a block of `b`'s own
- C.** Nothing: `b` is another name for `a`
- D.** Only the pointer; `mySize` and `myCapacity` stay 0

One Block, Two Owners

1

```
PyList b =  
a;
```

copies the three members, address and all.

THE STACK

PyList a;	
mySize	2
myCapacity	2
myArray	0x1A40

PyList b;	
mySize	2
myCapacity	2
myArray	0x1A40

THE HEAP

0x1A40



PyList b = a;

the same address, not a second block

One Block, Two Owners

2 `b[0] = 9;` changes what `a[0]` reads.

THE STACK

PyList a;	
mySize	2
myCapacity	2
myArray	0x1A40

PyList b;	
mySize	2
myCapacity	2
myArray	0x1A40

THE HEAP

0x1A40



`b[0] = 9;`
and `a[0]` reads 9

`PyList b = a;`
the same address, not a second block

One Block, Two Owners

3 **b** is destroyed, and frees the block.

THE STACK

PyList a;	
mySize	2
myCapacity	2
myArray	0x1A40

PyList b;	
mySize	2
myCapacity	2
myArray	0x1A40

*PyList b = a;
the same address, not a second block*

THE HEAP

0x1A40



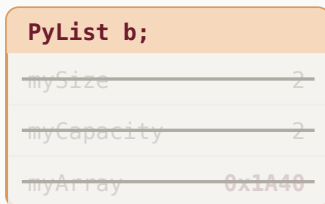
*b[0] = 9;
and a[0] reads 9*

*b is destroyed: delete [] myArray
the block is given back*

One Block, Two Owners

4 **a** is destroyed, and frees it again.

THE STACK



*PyList b = a;
the same address, not a second block*

THE HEAP

0x1A40



*b[0] = 9;
and a[0] reads 9*

*b is destroyed: delete [] myArray
the block is given back*

a is destroyed: delete [] myArray, again

CRASH

free(): double free detected

The default copy duplicated the **address**, not the data.
That is a **shallow copy**, and it goes wrong twice.

TALK TO YOUR NEIGHBOR · TTYN

What does this print, and what happens at line 6?

```
PyList a;           // 1
a.append(3);         // 2
PyList b = a;        // 3
b[0] = 9;            // 4
cout << a[0] << endl; // 5
// 6 a and b both go out of scope
```

- A.** 3, and each object frees its own block
- B.** 9, and the second destructor frees a block that is already gone
- C.** 3, and one block is leaked
- D.** 9, and line 3 does not compile

Deep Copy: the Copy Constructor

copy constructor

A constructor whose one parameter is **another object of the same class**. C++ calls it whenever a new object is built from an existing one. Write none and C++ supplies one that copies the address; write one and you decide what copying means.

```
PyList::PyList(const PyList &orig) {    // orig is the one we copy FROM
    mySize      = orig.mySize;          // same count
    myCapacity  = orig.myCapacity;      // same room
    myArray     = new Item[myCapacity]; // our OWN block, a new address
    for (unsigned i = 0; i < mySize; ++i) {
        myArray[i] = orig.myArray[i];  // copy the values across
    }
}
```

Allocate, copy, own: the same three moves as growing. Now each object frees its own block.

Three Times It Copies

```
PyList b = a;                                // 1. explicit copy

void show(PyList v);                          // 2. pass-by-value
show(a);

PyList makeList() {                           // 3. return by value
    PyList local;
    return local;
}
```

Case 2 is why week 1 said *pass big objects by const reference*. Every by-value call to `show()` copies the whole heap block.

Returning a Reference: `operator[]`

```
Item& PyList::operator[](unsigned i) {  
    return myArray[i];           // a reference, not a copy  
}  
  
v[2] = 99;           // works because v[2] IS the slot
```

Return `Item` and the call hands back a copy, and `v[2] = 99` does not compile at all. Returning `Item&` makes the call another name for the element.

The `this` Pointer

Python

```
def set_first(self, v):  
    self.my_first = v
```

C++

```
void setFirst(Item v) {  
    this->myFirst = v;  
}
```

C++ passes the object silently as `this`: a pointer to the object the method was called on. You rarely write it, but it's there, and `this->x` is just `(*this).x`.

Week 04 Recap

- `new` gets heap memory at runtime; every `new` needs one `delete`
- **size** vs **capacity** is what makes append cheap most of the time
- Growing means allocate, copy, delete, repoint: linear, so we double
- If your class calls `new`, it needs a **destructor**
- The default copy is **shallow**: aliasing plus a double delete
- Write a copy constructor that allocates its own block

Next: a05 →

